

Trabajo Final Integrador (TFI)

Desarrollo de Aplicaciones Web

“Del Código a la Arquitectura: El Criterio del Desarrollador”

TrainerOS

Plataforma SaaS de gestión para entrenadores personales

Autores — Grupo VOX: Conrado Gómez, Nicolás Zyngale, Matías Saravia, Lautaro Naglieri, Pablo Czurylo, Leandro Gramajo
Junio 2026

Índice

Fase 1 — Arquitectura Base y Esqueleto (El MVP)
Fase 2 — Evolución Forzada a Microservicios
Fase 3 — Mejora Incremental (Robustez y Escalabilidad)
Fase 4 — Infraestructura, DevOps y Ambientes
Fase 5 — Extensión de Próxima Generación: Integración con IA
Resumen de la Evolución y Entregables

Fase 1: Arquitectura Base y Esqueleto (El MVP)

1.1. Definición de la Temática y Modelo de Negocio

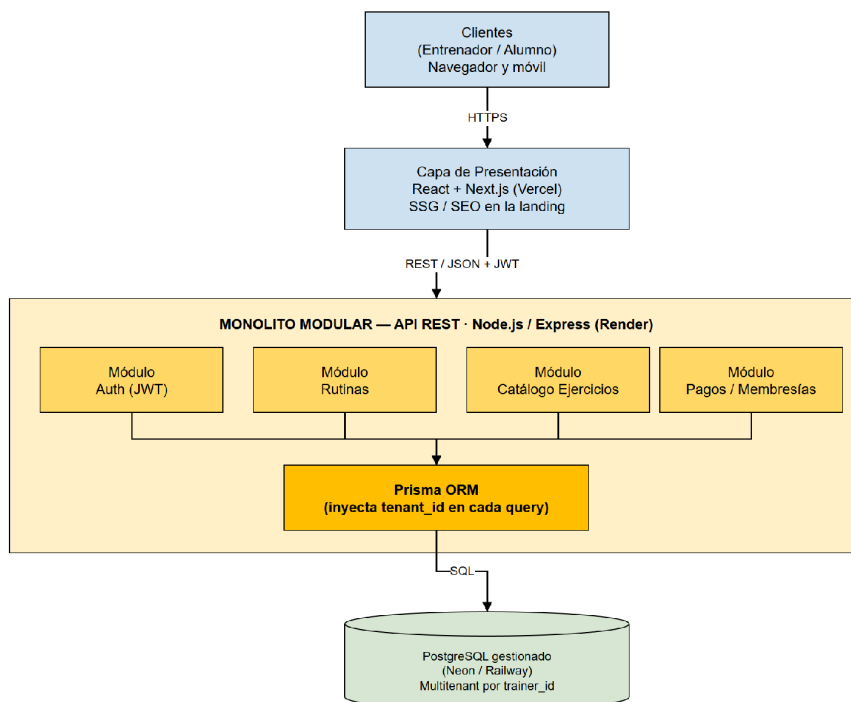
TrainerOS es una plataforma SaaS B2B2C orientada a entrenadores personales independientes. El problema que ataca es concreto: hoy los entrenadores gestionan decenas de alumnos con planillas de Excel y mensajes de WhatsApp, lo que genera caos logístico, rutinas desactualizadas y pérdida de control sobre los cobros de membresías.

Modelo de negocio: suscripción mensual por entrenador (modelo SaaS multitenant). El entrenador paga por la herramienta; sus alumnos acceden gratis a un panel móvil donde consumen la rutina del día y marcan su progreso. Las funcionalidades del MVP se limitan a: autenticación con Google, constructor de rutinas a partir de un catálogo de ejercicios, panel móvil del alumno y control de pagos de membresías con alertas de vencimiento.

1.2. Planteo Arquitectónico Inicial: Monolito Modular en 3 Capas

Para el volumen inicial (primeros 50 a 100 entrenadores, menos de 5.000 alumnos) se propone un monolito modular con arquitectura en capas (3-tier):

- **Capa de Presentación:** React.js para los paneles privados (entrenador y alumno) y Next.js con generación estática (SSG) para la landing pública, optimizando SEO para la adquisición orgánica. Desplegada en Vercel.
- **Capa de Aplicación:** API REST en Node.js + Express, organizada internamente en módulos con límites claros (Auth, Rutinas, Catálogo, Pagos). Valida JWT y aplica las reglas de negocio. Prisma ORM como capa de abstracción de datos. Desplegada en Render/Railway.
- **Capa de Datos:** PostgreSQL gestionado. El modelo relacional es mandatorio por la interconexión de entidades (Entrenador, Alumno, Rutina, RutinaEjercicio, Membresía) y el aislamiento multitenant: toda consulta inyecta el `trainer_id` validado desde el JWT.

Fase 1 — Monolito Modular en 3 Capas (MVP)*Figura 1 – Arquitectura inicial: monolito modular en 3 capas*

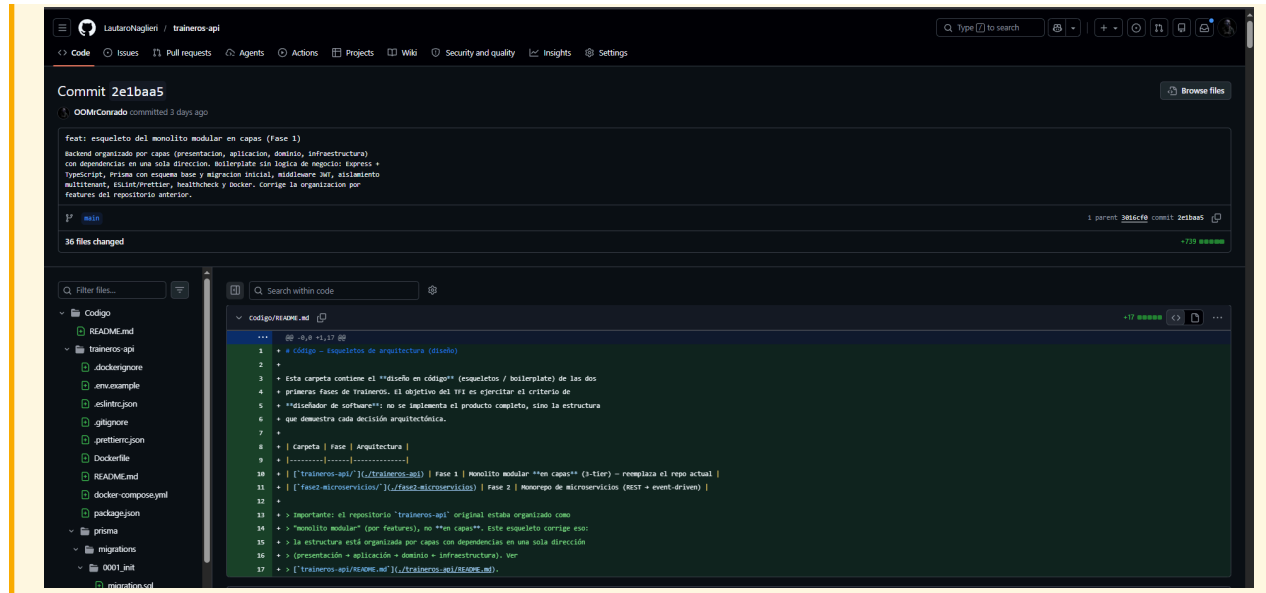
1.3. Justificación Técnica y de Negocio

¿Por qué esta arquitectura es suficiente para arrancar? Porque minimiza las dos variables críticas de un MVP: costo y tiempo de salida al mercado. Un solo despliegue, una sola base de datos y un solo pipeline permiten iterar en sprints semanales con un costo fijo de USD 7–16/mes. Node.js se elige por el ecosistema compartido con el frontend (un solo lenguaje, TypeScript de punta a punta), su excelente rendimiento en cargas I/O-bound (este sistema es 95% lecturas/escrituras de datos, sin cómputo pesado) y la velocidad de desarrollo con Express + Prisma. La modularidad interna no es accidental: los módulos del monolito están diseñados como futuros candidatos a microservicios, lo que reduce el costo de la migración de la Fase 2.

¿Cuáles son sus límites? (a) Escalado solo vertical: si el módulo de Rutinas recibe picos, hay que escalar todo el monolito; (b) un deploy defectuoso de cualquier módulo tumba el sistema completo (acoplamiento de despliegue); (c) la base única concentra el riesgo: una query pesada de reportes degrada las operaciones transaccionales; (d) un solo equipo puede trabajar cómodo, pero con varios equipos los merge conflicts y la coordinación de releases se vuelven un cuello de botella; (e) Render/Railway no ofrecen autoscaling granular ni SLAs de alta disponibilidad.

1.4. Artefactos Requeridos

- **Diagrama de arquitectura inicial:** Figura 1.
- **Repositorio Git:** GitHub Flow extendido con dos ramas permanentes (main = producción, dev = integración) y ramas temporales feat/, fix/ y chore/. El esqueleto inicial incluye únicamente: configuración de Express + TypeScript, Prisma con el esquema base y migración inicial, middleware JWT vacío, ESLint/Prettier y healthcheck. Sin lógica de negocio.





Commits del esqueleto del monolito en capas — Fase 1 (commit 2e1baa5) y estructura de las 3 capas. Repositorio: <https://github.com/LautaroNaglieri/traineros-api>

Fase 2: Evolución Forzada a Microservicios

2.1. El Cambio de Contexto (Escenario de Negocio)

Doce meses después del lanzamiento, TrainerOS firma convenios B2B con dos cadenas regionales de gimnasios que incorporan a sus entrenadores a la plataforma. El sistema pasa de 100 entrenadores a **5.000 entrenadores y ~120.000 alumnos activos**. Aparecen tres presiones que el monolito no resuelve:

- **Picos de tráfico predecibles y violentos:** entre las 18 y las 21 h los alumnos consultan su rutina desde el gimnasio (lecturas masivas), mientras los entrenadores cargan planificaciones. Los lunes el pico se triplica.
- **Equipos independientes:** La empresa crece a tres equipos de desarrollo (Core de rutinas, Pagos/B2B, Experiencia del alumno) que necesitan desplegar sin coordinarse entre sí.
- **Requisitos B2B:** las cadenas exigen SLA de disponibilidad, facturación consolidada y notificaciones masivas (recordatorios de vencimiento a miles de alumnos), cargas que no pueden competir por recursos con el flujo crítico de consulta de rutinas.

2.2. Paso A: Microservicios Tradicionales (REST síncrono)

La primera descomposición separa el monolito por capacidades de negocio, siguiendo los límites modulares ya definidos en la Fase 1:

- **MS Auth & Tenants:** identidad, JWT, aislamiento multitenant.
- **MS Rutinas:** planificación y asignación (núcleo del negocio).
- **MS Catálogo:** ejercicios globales, casi 100% lecturas.
- **MS Pagos/Membresías:** cobros, vencimientos, facturación B2B.
- **MS Notificaciones:** push, email y WhatsApp.

Cada servicio expone una API REST detrás de un **API Gateway** (ruteo, rate limiting, validación JWT centralizada) y aplica el patrón **Database per Service** para garantizar autonomía de despliegue y de esquema. La comunicación entre servicios es HTTP/REST síncrona.

TFI – Desarrollo de Aplicaciones Web · TrainerOS

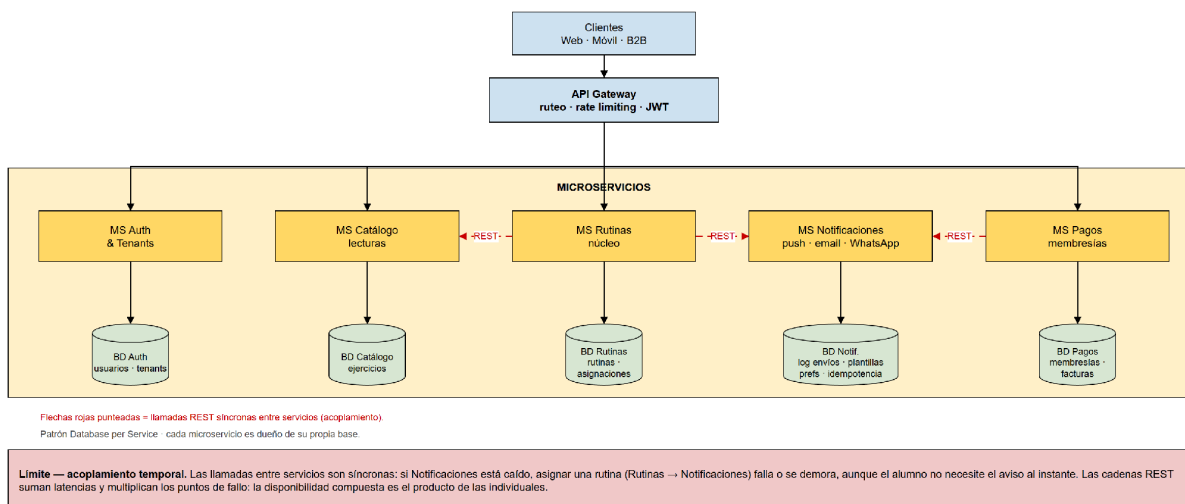
Fase 2 · Paso A — Microservicios Tradicionales (REST síncrono)

Figura 2 – Paso A: microservicios tradicionales con comunicación REST síncrona

Límite de este diseño: el acoplamiento temporal. Si Notificaciones está caído, la asignación de una rutina falla o se demora aunque el alumno no necesite el push de inmediato. Las llamadas en cadena (Rutinas → Catálogo → Notificaciones) suman latencias y multiplican los puntos de fallo: la disponibilidad compuesta es el producto de las disponibilidades individuales.

2.3. Paso B: Microservicios Modernos (Event-Driven)

La evolución natural es invertir la dependencia: los servicios dejan de llamarse entre sí y pasan a publicar hechos de negocio en un **Message Broker (Google Pub/Sub)**. Eventos principales: rutina.asignada, entrenamiento.completado, pago.confirmado, membresia.por_vencer.

TFI – Desarrollo de Aplicaciones Web · TrainerOS

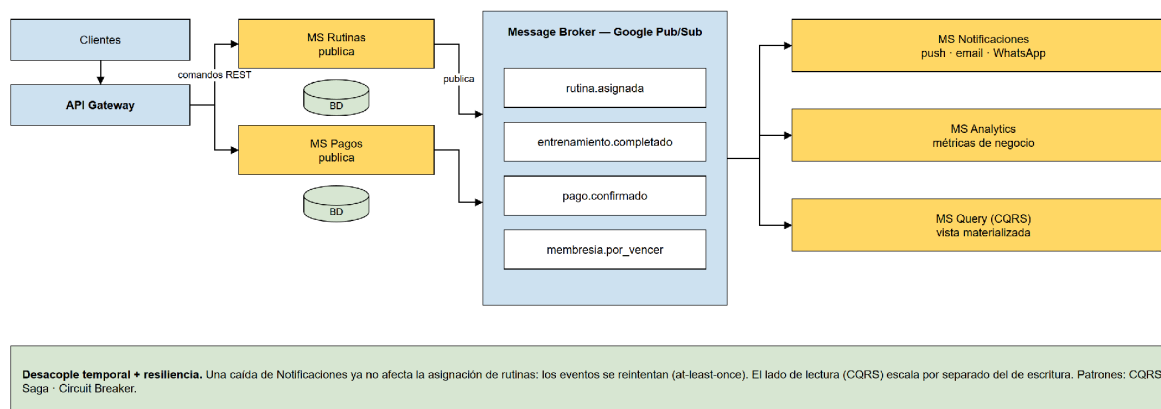
Fase 2 · Paso B — Microservicios Modernos (orientados a eventos)

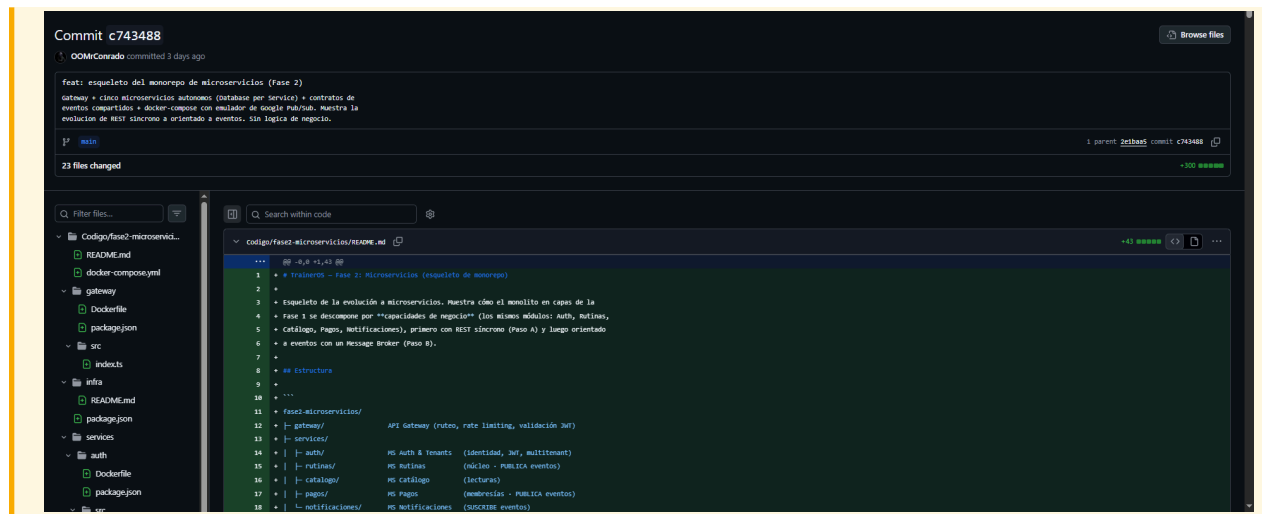
Figura 3 – Paso B: arquitectura orientada a eventos con Pub/Sub

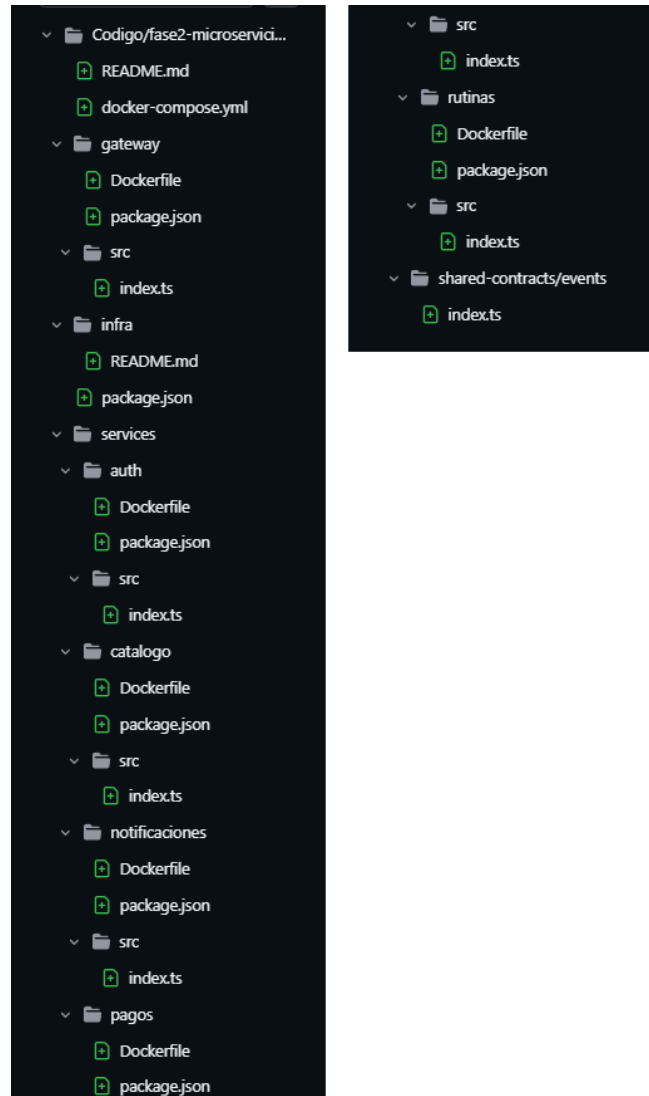
Patrones aplicados y su justificación

Patrón	Dónde se aplica	Por qué
API Gateway	Punto único de entrada (Kong / Cloud Endpoints)	Centraliza JWT, rate limiting y versionado; los clientes no conocen la topología interna.
Database per Service	Cada microservicio con su propia base de datos	Autonomía de esquema y despliegue; evita el acoplamiento por datos.
Pub/Sub (eventos)	Rutinas y Pagos publican; Notificaciones, Analytics y Query suscriben	Desacopla temporalmente: una caída de Notificaciones no afecta la asignación de rutinas; los eventos se reintentan.
CQRS	MS Rutinas (escritura) vs. MS Progreso/Query (lectura con vista materializada)	El patrón de carga es asimétrico: miles de alumnos leen su rutina, pocos entrenadores escriben. Se optimiza cada lado por separado.
Saga (orquestrada)	Flujo de cobro de membresía: intento de pago → confirmación → extensión de membresía → notificación; con compensaciones si un paso falla	No hay transacciones distribuidas entre bases separadas; la saga garantiza consistencia eventual con rollback de negocio.
Circuit Breaker	Llamadas síncronas residuales (ej. pasarela de pagos externa)	Evita el efecto dominó: ante fallos repetidos abre el circuito y responde fallback en lugar de agotar recursos.

2.4. Artefactos Requeridos

- Diagramas de ambas arquitecturas: Figuras 2 y 3.





Commits del esqueleto del monorepo de microservicios — Fase 2 (commit c743488): gateway + 5 servicios, shared-contracts y docker-compose con Pub/Sub. Repositorio: <https://github.com/LautaroNaglieri/traineros-api>

Fase 3: Mejora Incremental (Robustez y Escalabilidad)

Con la base event-driven establecida, la Fase 3 prepara el sistema para alta disponibilidad y tolerancia a fallos a gran escala. Las decisiones se ordenan por capa:

3.1. Capa de Caché (Redis)

Se incorpora **Memorystore for Redis** con estrategia cache-aside. Qué se cachea y por qué: (a) el catálogo de ejercicios (global, casi inmutable, TTL 24 h) — elimina la carga de lectura más repetida del sistema; (b) la “rutina del día” por alumno (TTL 1 h, invalidación al recibir rutina.asignada) — absorbe el pico de las 18–21 h; (c) sesiones y contadores de rate limiting del Gateway. Con esto, la mayoría de las lecturas del pico nunca llegan a PostgreSQL.

3.2. Replicación de Base de Datos (Read Replicas)

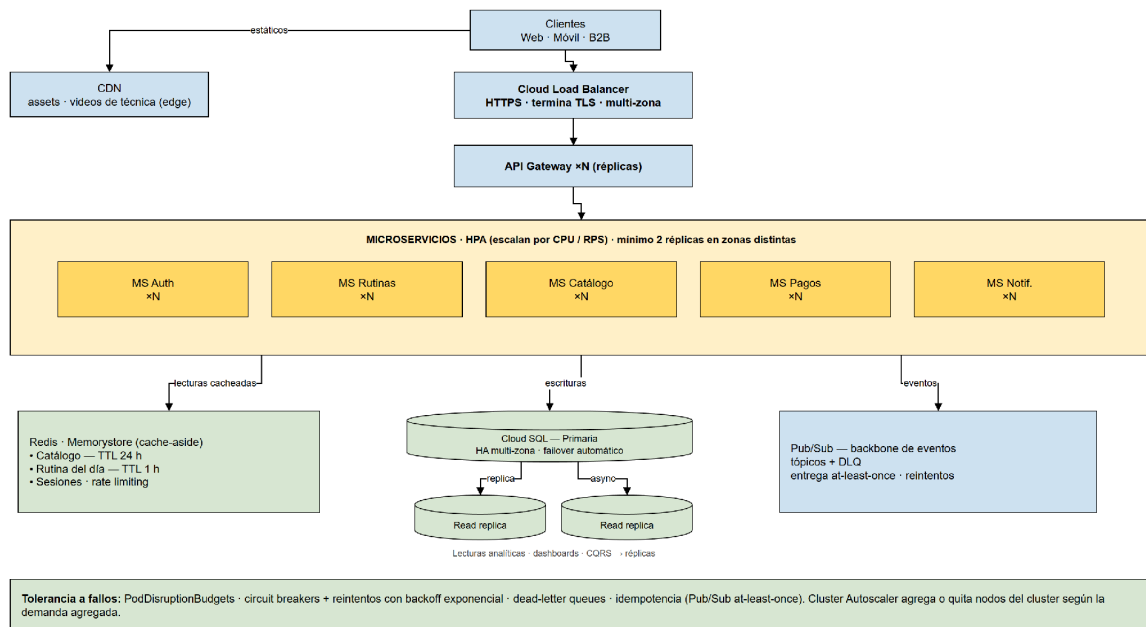
Cloud SQL para PostgreSQL en alta disponibilidad (primaria multi-zona con failover automático) más read replicas. Las escrituras van a la primaria; los dashboards, reportes y el servicio de Query (CQRS) leen de las réplicas. Se asume replicación asíncrona y consistencia eventual en lecturas analíticas, aceptable para el negocio (un dashboard con segundos de retraso no afecta el entrenamiento).

3.3. Balanceo de Carga y Auto-scaling

- **Cloud Load Balancer (HTTPS, multi-zona):** distribuye el tráfico entre las réplicas del Gateway y termina TLS.
- **HPA (Horizontal Pod Autoscaler):** cada microservicio escala por CPU/RPS; Rutinas y Query escalan agresivamente en el pico de la tarde y se contraen de madrugada.
- **Cluster Autoscaler:** agrega/quita nodos del cluster según demanda agregada, controlando el costo.
- **CDN:** assets estáticos y videos de técnica del catálogo se sirven desde el edge, no desde los pods.

3.4. Tolerancia a Fallos

- Mínimo 2 réplicas por servicio en zonas distintas; **PodDisruptionBudgets** para sobrevivir mantenimientos.
- **Circuit breakers y reintentos con backoff exponencial** en toda llamada síncrona; timeouts explícitos en cada salto.
- **Colas con dead-letter:** los eventos que fallan repetidamente van a una dead-letter queue para reproceso manual, sin frenar el flujo.
- **Idempotencia:** los consumidores procesan eventos duplicados sin efectos secundarios (Pub/Sub garantiza at-least-once).

Fase 3 — Robustez y Escalabilidad (caché · read replicas · LB · auto-scaling)*Figura 4 – Arquitectura escalada: LB, auto-scaling, caché y read replicas*

Fase 4: Infraestructura, DevOps y Ambientes

Nube elegida: Google Cloud Platform (GCP). Justificación: GKE es el Kubernetes gestionado de referencia (Google es el creador de K8s), Pub/Sub ya es el broker de la arquitectura (servicio nativo, serverless y sin administración de cluster, a diferencia de operar Kafka), Cloud SQL cubre PostgreSQL con HA y replicas en un clic, y el modelo de proyectos de GCP simplifica el aislamiento entre ambientes.

4.1. Estructura de Ambientes

Ambiente	Propósito	Infraestructura	Datos
Desa	Desarrollo local y sandbox de integración	docker-compose local (servicios + Postgres + emulador Pub/Sub); sandbox compartido en GKE (namespace desa) con recursos mínimos	Datos sintéticos / seeds
Testing/QA	Pruebas funcionales y de regresión automáticas	Proyecto GCP propio; GKE reducido (1 nodo spot); Cloud SQL mínima	Dataset controlado y reseteable
UAT	Smoke tests y validación pre-productiva con usuarios clave	Proyecto GCP propio; espejo de Prod a escala reducida (mismas versiones, menos réplicas)	Copia anonimizada de producción
Prod	Entorno vivo con entrenadores y alumnos reales	Proyecto GCP dedicado; GKE multi-zona, Cloud SQL HA + replica, Redis, monitoreo y alertas	Datos reales, backups automáticos + PITR

Cada ambiente vive en un proyecto GCP separado (aislamiento de permisos IAM, redes y facturación). La promoción entre ambientes es siempre del mismo artefacto: la imagen Docker inmutable etiquetada por commit SHA; lo único que cambia es la configuración inyectada.

4.2. Estrategia de Branching y Mapeo con Ambientes

Se mantiene el **GitHub Flow extendido** definido en el MVP (main + dev + ramas feat/fix/chore), formalizado con Pull Requests obligatorios y madurado para mapear los cuatro ambientes:

Rama / Evento Git	Ambiente y acción
feat/* · fix/* (PR abierto)	CI: lint + tests + build. Preview deploy efímero del frontend (Vercel).
Merge a dev	Despliegue automático a Testing/QA + suite de regresión e2e.
Tag rc-* sobre dev	Despliegue automático a UAT + smoke tests; validación de usuarios clave.
PR dev → main (aprobado) + tag v*	Despliegue a Producción con aprobación manual (environment protegido) y rollout progresivo.

Se elige GitHub Flow extendido por sobre GitFlow completo: el equipo despliega varias veces por semana y no mantiene múltiples versiones en paralelo, por lo que las ramas release/ y hotfix/ permanentes de GitFlow agregarían burocracia sin beneficio. Trunk-Based puro se descarta porque el equipo aún necesita la rama dev como buffer de integración para QA.

4.3. Pipelines CI/CD (GitHub Actions)

Herramienta: GitHub Actions — integración nativa con el repositorio, runners gestionados, environments con aprobaciones manuales y secretos por ambiente, sin operar un Jenkins propio. Etapas del pipeline:

- **CI (en cada PR):** lint → tests unitarios → tests de integración (Postgres + Pub/Sub emulador en contenedores de servicio) → build de imagen Docker → scan de vulnerabilidades (Trivy).
- **Publicación:** push de la imagen a Artifact Registry etiquetada con el SHA del commit.
- **CD:** deploy a QA (automático), UAT (tag rc-*) y Prod (aprobación manual) vía kubectl/Helm contra GKE. Rollout progresivo con rollback automático si fallan los healthchecks.

4.4. Contenedores y Orquestación (Docker + Kubernetes)

- **Docker:** una imagen por microservicio (build multi-stage de Node.js, imagen final distroless ~80 MB). La misma imagen corre en los cuatro ambientes.
- **Deployments:** uno por servicio, con réplicas mínimas (2 en Prod), readiness/liveness probes y límites de CPU/memoria.
- **Services + Ingress:** Services ClusterIP para tráfico interno; Ingress (GCE) expone el Gateway al Cloud Load Balancer.
- **HPA:** escala los pods por CPU/RPS (Fase 3); ConfigMaps y Secrets inyectan la configuración por ambiente.
- **Namespaces:** separación lógica por ambiente dentro de los clusters no productivos para optimizar costos.

4.5. Infraestructura como Código (Terraform)

Toda la infraestructura se define en Terraform con módulos reutilizables y un directorio por ambiente. El estado remoto vive en un bucket de GCS con bloqueo. Estructura:

```
infra/
├── modules/
│   ├── network/      (VPC, subredes, Cloud NAT, firewall)
│   ├── gke/          (cluster, node pools, autoscaling)
│   ├── cloudsql/     (instancia, HA, replicas, backups)
│   ├── redis/        (Memorystore)
│   ├── pubsub/       (tópicos, suscripciones, DLQs)
│   └── iam/           (service accounts, roles mínimos)
└── environments/
    └── qa/            main.tf + terraform.tfvars (tamaños mínimos)
```

```

├── uat/      main.tf + terraform.tfvars (espejo reducido)
├── prod/     main.tf + terraform.tfvars (HA completa)
└── backend.tf (estado remoto en GCS)

```

Los mismos módulos se instancian con variables distintas por ambiente (cantidad de nodos, tiers de máquinas, flags de HA). Un cambio de infraestructura sigue el mismo flujo que el código: PR → terraform plan en CI → aprobación → terraform apply.

4.6. Estimación de Costos y Servidores (Producción, GCP)

Componente	Servicio GCP	Cantidad	Rol en la Arquitectura	Costo Mensual Aprox (USD)
Gestión del cluster	GKE (fee de administración)	1 cluster	Orquestación	\$0 (crédito free tier cubre 1 cluster)
Nodos de cómputo	GKE – e2-standard-2 (2 vCPU / 8 GB)	3 nodos	Pods de microservicios, gateway y workers	\$150
Base de datos	Cloud SQL PostgreSQL (2 vCPU / 8 GB, HA)	1 primaria + 1 read replica	Persistencia transaccional y lecturas	\$250
Caché	Memorystore for Redis (1 GB, Basic)	1 instancia	Caché de catálogo, rutinas y sesiones	\$35
Message Broker	Pub/Sub	6 tópicos (4 de negocio + 2 de IA) + DLQs	Backbone de eventos	\$10
Balanceador	Cloud Load Balancing (HTTPS)	1	Entrada de tráfico multi-zona	\$20
Registro de imágenes	Artifact Registry + Storage	1 repo	Imágenes Docker y backups	\$10
Observabilidad	Cloud Monitoring + Logging	—	Métricas, logs, alertas	\$25
Frontend	Vercel Pro	1 plan	Hosting React/Next.js + CDN edge	\$20
TOTAL Producción				≈ \$520 / mes

QA + UAT suman ≈ USD 150/mes adicionales usando nodos spot, instancias mínimas de Cloud SQL y apagado nocturno automático. **Lectura de negocio:** con 5.000 entrenadores pagando una suscripción promedio de USD 15, el costo total de infraestructura (~USD 670) representa menos del 1% del MRR, validando la sostenibilidad del diseño. Los valores son aproximados (región us-central1, precios de lista sin descuentos por uso comprometido).

Fase 5: Extensión de Próxima Generación – Integración con IA (Opción A: APIs de LLM)

5.1. Casos de Uso de Negocio

- **Borrador de rutina asistido:** el entrenador pide “rutina de hipertrofia 4 días para alumno intermedio con molestia en hombro”; el LLM genera un borrador usando el catálogo y el historial del alumno, que el entrenador edita y aprueba. La IA asiste, nunca publica sola.
- **Resumen de progreso:** síntesis mensual automática del avance de cada alumno (adherencia, cargas, feedback) para que el entrenador prepare la renovación de membresía.
- **Clasificación de feedback:** los comentarios de alumnos (texto libre) se clasifican automáticamente (dolor/lesión → alerta urgente al entrenador; motivación; logística) alimentando el tablero de gestión.

5.2. Arquitectura de Integración: Asincronismo y Timeouts

Principio rector: una llamada a un LLM externo (Anthropic Claude / OpenAI) tarda entre 2 y 60 segundos y puede fallar; jamás debe bloquear un request HTTP del usuario ni acoplar la disponibilidad de TrainerOS a la del proveedor. Por eso la integración reutiliza el backbone event-driven:

- **Patrón de respuesta diferida:** el cliente recibe 202 Accepted + job_id de inmediato; el MS Asistente IA encola el trabajo en el tópico ia.jobs de Pub/Sub.
- **Workers dedicados en GKE:** consumen la cola, arman el prompt con el contexto del alumno (historial, catálogo permitido, restricciones) y llaman a la API del LLM. Escalan con HPA según la profundidad de la cola, sin competir con el tráfico transaccional.
- **Manejo de timeouts y fallos:** timeout explícito de 60 s por llamada; reintentos con backoff exponencial y jitter (máx. 3); circuit breaker sobre el proveedor con fallback a un segundo proveedor o degradación elegante (“el asistente no está disponible, intentá más tarde”); idempotencia por job_id para evitar dobles cobros de tokens; dead-letter queue para jobs irrecuperables.
- **Entrega del resultado:** el worker persiste la respuesta, publica ia.job.terminado y el MS Notificaciones avisa al cliente por websocket/push. El frontend también puede consultar GET /ai/jobs/{id} (polling de respaldo).
- **Control de costos y datos:** presupuesto de tokens por tenant con límites mensuales, caché de respuestas para prompts repetidos (clasificación), y anonimización de datos personales del alumno antes de salir hacia la API externa.

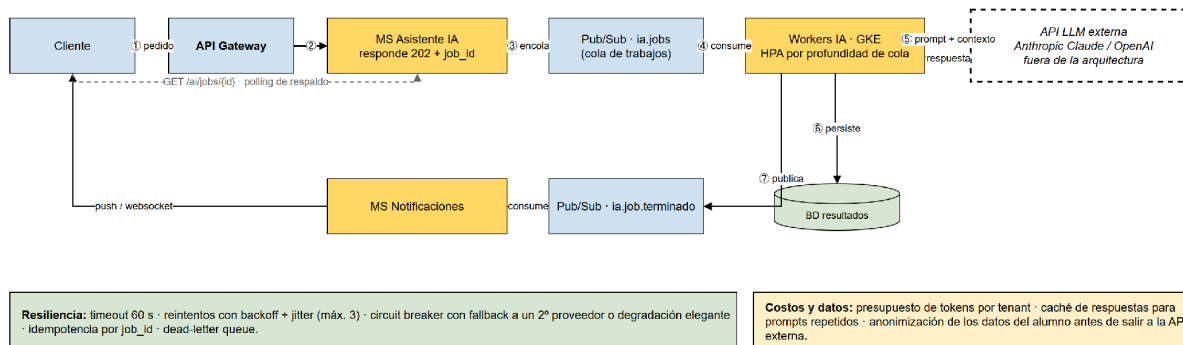
Fase 5 — Integración con IA (Opción A: APIs de LLM) · flujo asíncrono

Figura 5 – Flujo asíncrono de integración con APIs de LLM

5.3. Justificación Técnica y de Negocio

Se elige la Opción A (consumo directo de APIs de LLM) porque los tres casos de uso operan sobre datos estructurados que ya viven en PostgreSQL (historial, catálogo, membresías) y caben en el contexto del prompt; no se necesita todavía una base vectorial. Esta decisión minimiza piezas nuevas de infraestructura y reutiliza patrones existentes (colas, workers, circuit breakers). Si a futuro el catálogo crece a miles de ejercicios con fichas técnicas extensas y documentos de los entrenadores, la evolución natural es incorporar RAG con una base vectorial, sin cambiar el esqueleto asíncrono ya diseñado: solo se agrega una etapa de retrieval previa al armado del prompt.

Resumen de la Evolución y Entregables

Evolución arquitectónica en una mirada

Fase	Arquitectura	Disparador de negocio	Costo infra aprox.
1	Monolito modular 3 capas (Vercel + Render + Postgres)	Validar el MVP con 50–100 entrenadores	\$7–16 / mes
2	Microservicios REST → Event-Driven (Pub/Sub)	B2B: 5.000 entrenadores, 120.000 alumnos, equipos independientes	intermedio
3	+ Redis, read replicas, LB, auto-scaling	Picos 18–21 h, SLA de disponibilidad	incluido en Fase 4
4	GKE + Terraform + CI/CD, 4 ambientes en GCP	Operación profesional y reproducible	≈ \$520 prod + \$150 no-prod
5	+ Workers de IA contra APIs de LLM	Diferenciación de producto: asistente del entrenador	variable por tokens